

Introducción a los Sistemas Lógicos y Digitales

Trabajo Integrador

Grupo 11

Integrantes:

- Ariza, Ariana Magali, 02876/6
- Lopes, Lautaro Nahuel, 02978/9
- Ruiz Diaz, Lautaro, 03420/6
- Savran, Alexis Pablo, 03305/4

Descripción

Se desea implementar un circuito que implemente un registro de 12 bits, que pueda inicializarse externamente, y cuyo valor inicial se pueda incrementar o decrementar hasta alcanzar otro valor también programable externamente.

El comportamiento de incremento o decremento se seleccionará mediante una entrada UP/DOWN.

El circuito tendrá una entrada/salida bidireccional de 12 bits DATA[11..0], por las cuales se introducirán los valores inicial y final. La carga de estos valores deberá realizarse mediante la activación de dos señales, LOAD_I y LOAD_F.

El incremento que se realice en la cuenta en cada ciclo de reloj, será introducido por la entrada b2..b0 (3 bits) que codificará los valores de 0 a 7.

Cuando el valor del incremento sea 0, el circuito deberá activar la línea de salida PAUSA.

Los valores que adopte el contador deberán estar disponibles en DATA[11..0].

Cuando el circuito llegue al valor final deberá activar la línea de salida FIN durante un ciclo de reloj y reiniciar la cuenta o detenerse según el valor que esté presente en la línea de entrada CONTINUAR. si está en alto, reinicia la cuenta, si está en bajo la detiene.

Para cargar un nuevo juego de valores iniciales y finales, se debe activar la línea RECARGAR en 1, lo que detendrá el contador y permitirá el uso de las líneas LOAD_I y LOAD_F.

Además se tiene una entrada START, que inicia el funcionamiento del contador, y una salida ERROR, que se activa cuando los valores iniciales y finales no son consistentes con el modo de operación indicado por la entrada UP/DOWN.

En la Figura 1 a continuación se muestran un esquema con las Entradas y Salidas.

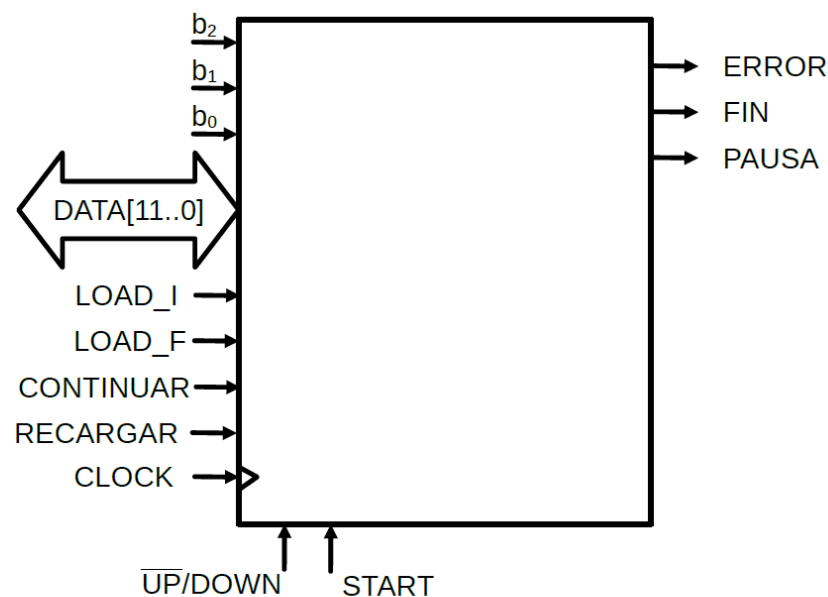


Figura 1

En la Figura 2, se muestra un esquema propuesto de implementación del contador utilizando registros y un sumador. A esto debe agregarse un comparador para determinar cuando se llega al valor final y además se debe verificar la consistencia entre el valor inicial, el valor final y la dirección de la cuenta (ascendente o descendente).

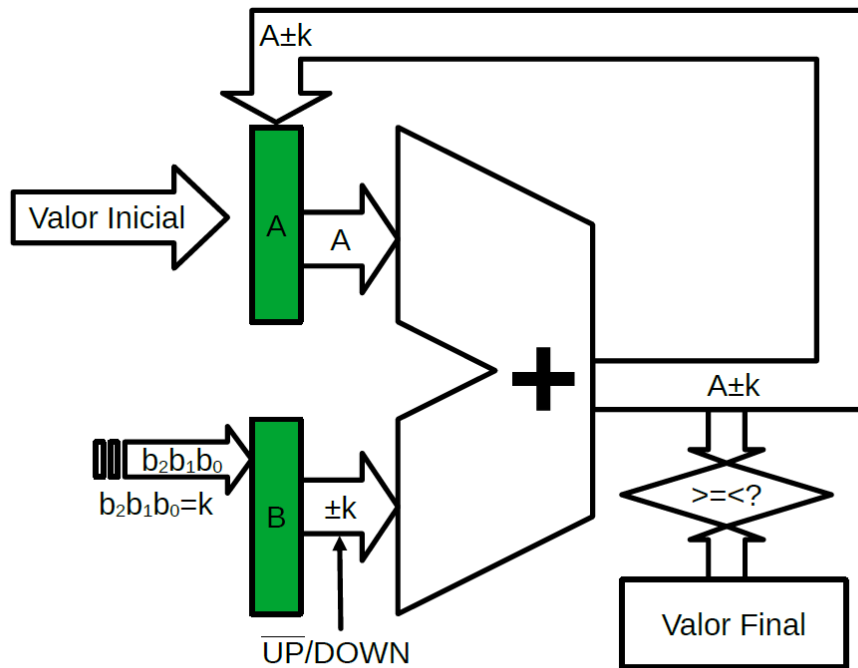


Figura 2

Desarrollo

Para cumplir con lo solicitado, el circuito requiere un contador síncrono que almacene y manipule valores de hasta 12 bits por lo que se necesitarán 12 flip-flops tipo D para implementar los registros. A su vez, se requiere almacenar 3 valores distintos (valor inicial, valor final y valor actual), por lo tanto, se utilizará un total de 36 flip-flops (3x12). Cada uno de estos flip-flops estarán sincronizados mediante una señal de reloj y habilitados por señales de control (LOAD_I, LOAD_F).

Los bloques generales que debemos desarrollar son:

- 3 registros (valor inicial, valor final y valor actual del contador)
- Sumador/restador
- Comparador
- Máquina de estados finitos

REGISTROS

Para los registros de valor inicial y final de 12 bits necesitamos:

- 12 flip-flops tipo D
- Un control de carga (LOAD_I y LOAD_F)
- Reloj (CLK)
- Entrada de datos: DATA[11..0]
- Salida: Q[11..0] (contenido del registro)
- 12 multiplexores

Componentes necesarios para cada bit:

- >Flip-Flop tipo D (1 bit)
- >Multiplexor 2 a 1 (MUX2) por bit
 - Para elegir entre mantener el valor actual (Q[i]) o cargar DATA[i]
- >Reloj común a todos los FF
- >Señal LOAD como selector de los MUX

Para 12 bits, hacemos 12 veces (de i=0 a i=11):

Insertar un MUX 2 a 1:

Entrada 0: se conecta a la salida del flip-flop (Q[i])

Entrada 1: DATA[i]

Selector: LOAD

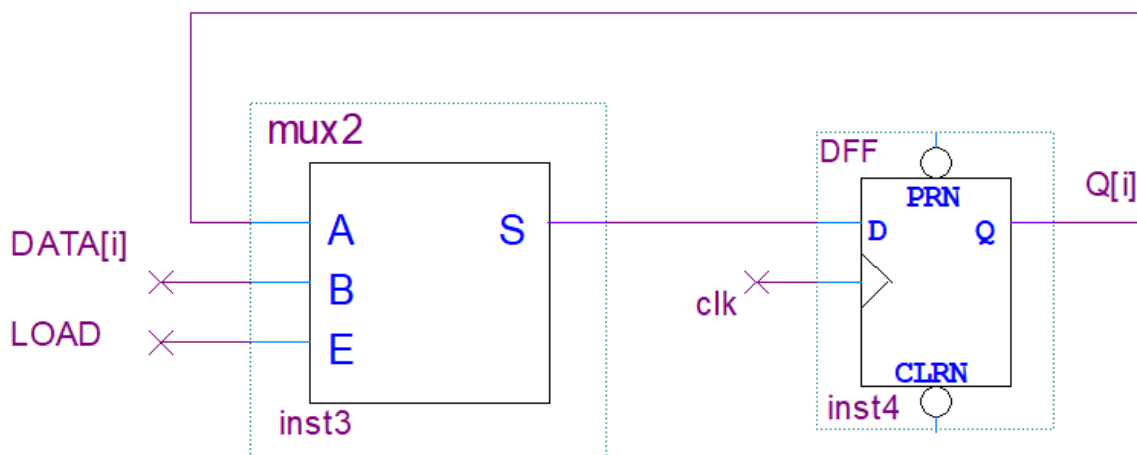
Salida: va a la entrada D del flip-flop

Insertar un FF tipo D:

Entrada D: del MUX

CLK: común

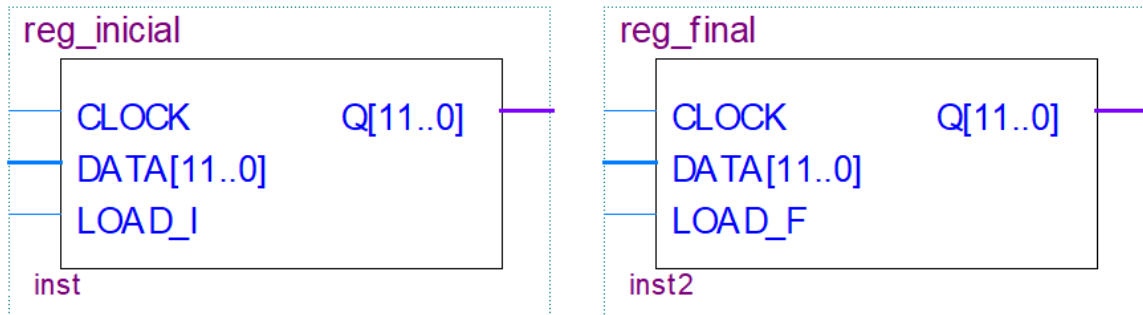
Salida Q: va al bus de salida y al MUX



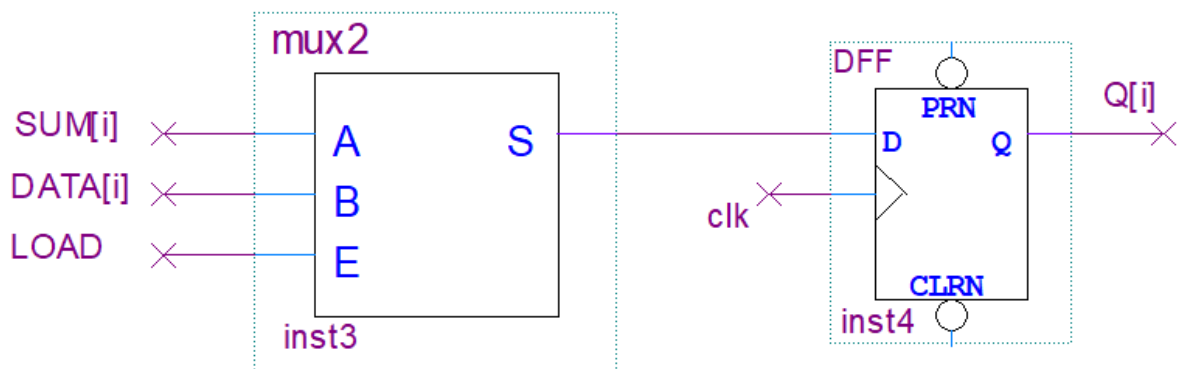
Si LOAD=1, el MUX deja pasar DATA[i] → se carga un nuevo valor.

Si LOAD=0, el MUX deja pasar Q[i] → mantiene el valor anterior.

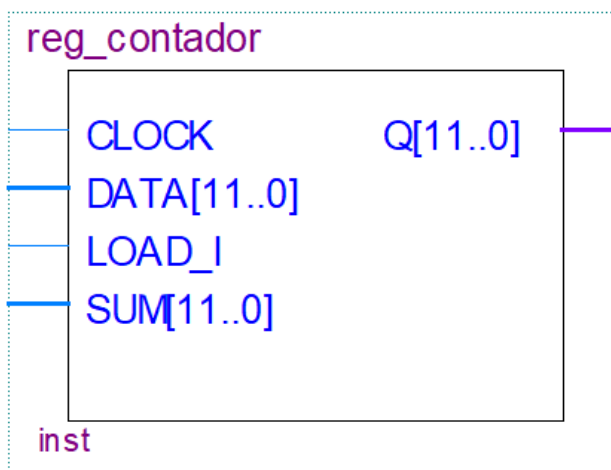
Los registros inicial y final son diseñados de la misma manera, la diferencia es la entrada LOAD.



Para el registro contador tendremos los mismos componentes que los registros anteriores pero ahora el multiplexor selecciona entre $DATA[i]$ y $SUM[i]$, y la señal de selección será $LOAD_I$.



si $LOAD = 1 \Rightarrow$ entra $DATA[i]$ al FF (carga inicial)
 si $LOAD = 0 \Rightarrow$ entra $SUM[i]$ al FF (sigue contando)



$Q[11..0]$: valor de cuenta
 $DATA[11..0]$: carga inicial
 $SUM[11..0]$: salida del sumador

SUMADOR / RESTADOR

Para el sumador usaremos un circuito que nos permita sumar y restar números mediante la entrada /UP/DOWN. En la resta se llevará el incremento b a su representación negativa en $Ca2$ y se sumará al contador.

El sumador de 12 bits lo hicimos encadenando 12 full adders de 1 bit.

Full Adder de 1 bit:

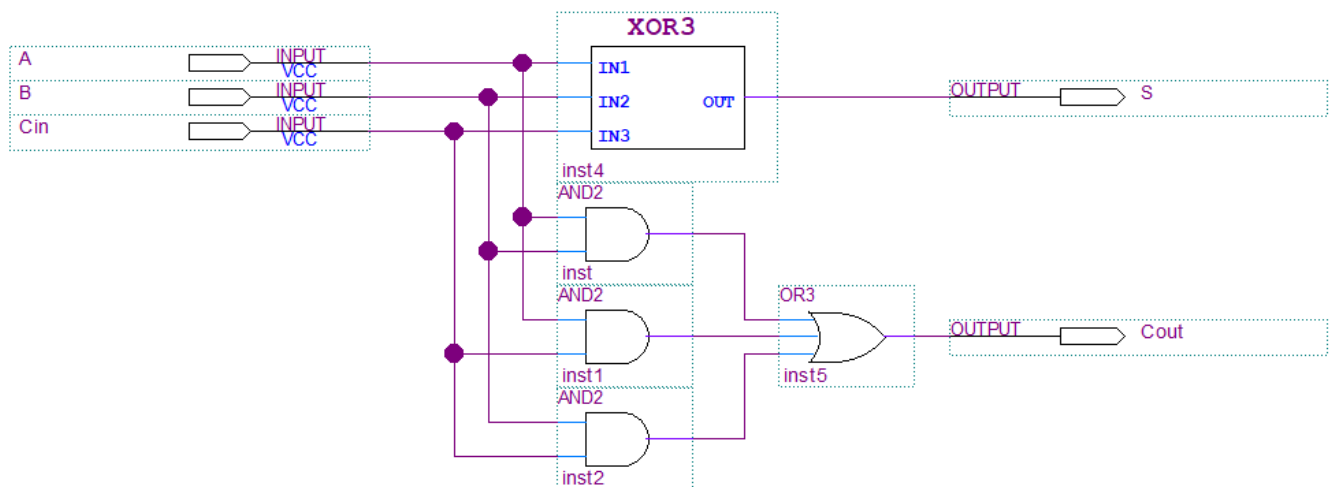
A	B	Cy_{IN}	S	Cy_{OUT}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

S	$\neg B/Cin$	$\neg BCin$	$BCin$	B/Cin
$\neg A$	0	1	0	1
A	1	0	1	0

$$S = A \oplus B \oplus Cin$$

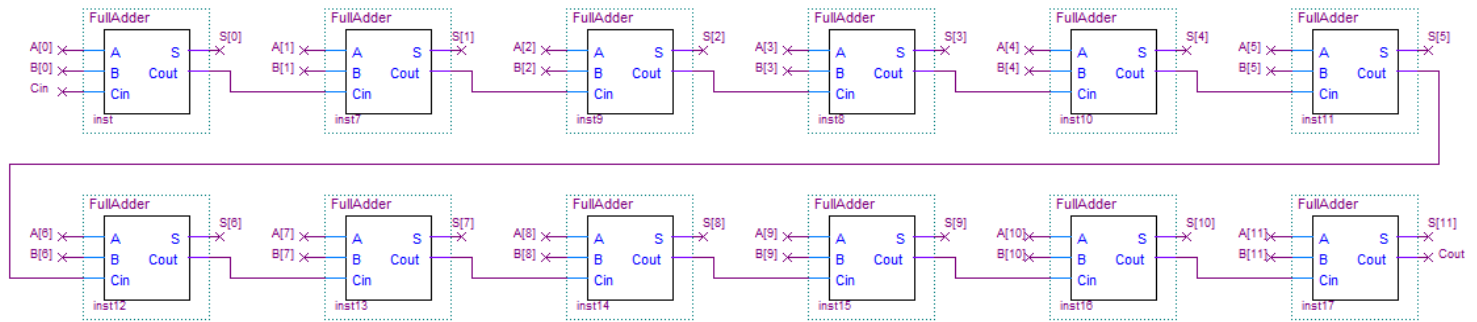
Cout	$\neg B/Cin$	$\neg BCin$	$BCin$	B/Cin
$\neg A$	0	0	1	0
A	0	1	1	1

$$Cout = AB + ACin + BCin$$



En el acarreo de entrada debemos poner la entrada UP/DOWN y el Cout de cada sumador va al Cin del siguiente.

Sumador de 12 bits:



Como debemos sumar un numero de 12 bits con uno de 3, debemos extenderlo agregando 0's a la izquierda.

Queremos que:

Si UP/DOWN = 0: se suma b al contador (A+b)

Si UP/DOWN = 1: se reste b al contador (A-b).

Recordando $\rightarrow Ca2(b) = \sim b + 1$

Esto implica:

Invertir cada bit de b si UP/DOWN = 1.

Sumar 1 si UP/DOWN = 1 (esto lo hacemos con el bit de acarreo inicial Cin=UP/DOWN).

Para cada bit i de b (de 0 a 11) $\rightarrow k[i] = b[i] \text{ XOR UP/DOWN}$

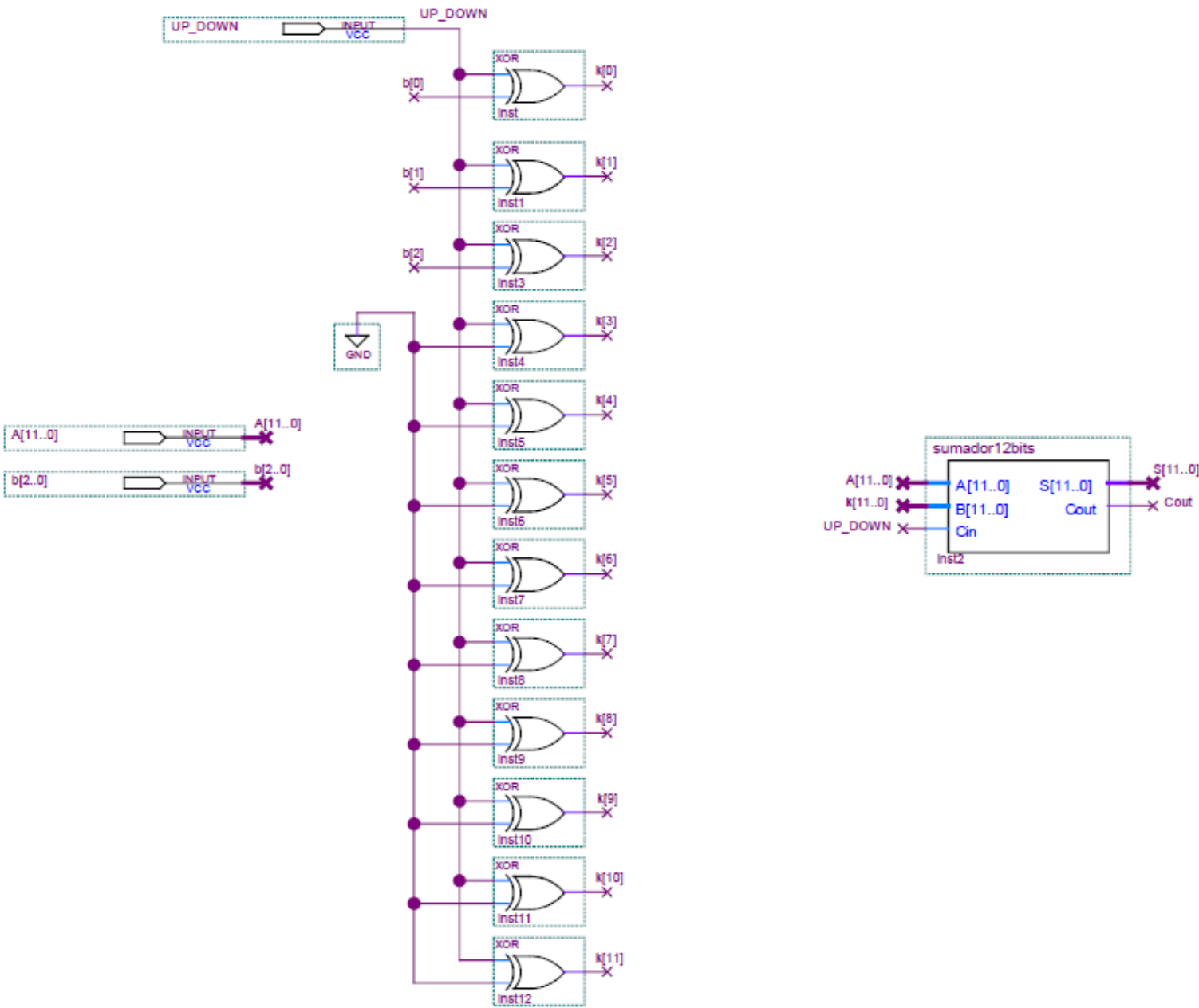
Esto funciona porque:

Si UP/DOWN = 0 $\rightarrow k[i] = b[i]$ (no se invierte)

Si UP/DOWN = 1 $\rightarrow k[i] = \sim b[i]$ (se invierte)

Así construimos $k[11..0]$, que es la entrada modificada que va al sumador de 12 bits.

Entonces el diseño del Sumador/restador quedaría de la siguiente manera:



A continuación se presentan los mapas de Karnaugh realizados para simplificar las funciones de algunas señales:

PAUSA	/B1/B0	/B1B0	B1B0	B1/B0
/B2	1	0	0	0

B2	0	0	0	0
----	---	---	---	---

$$\text{PAUSA} = \text{/B2} * \text{/B1} * \text{/B0}$$

Para la señal FIN necesitamos el diseño de un comparador que indique que

$$\text{valor_actual}(\text{Qc}) = \text{valor_final}(\text{Qf})$$

>Ambos valores son de 12 bits, por lo tanto necesitamos un comparador de 12 bits.

>Si son iguales, activamos la señal FIN durante 1 ciclo de reloj.

Comparamos cada bit de contador[i] con final[i], usando un XNOR:

$$\text{EQ}[i] = \text{Qc}[i] \text{ XNOR } \text{Qf}[i]$$

El resultado EQ[i] será 1 si ambos bits son iguales.

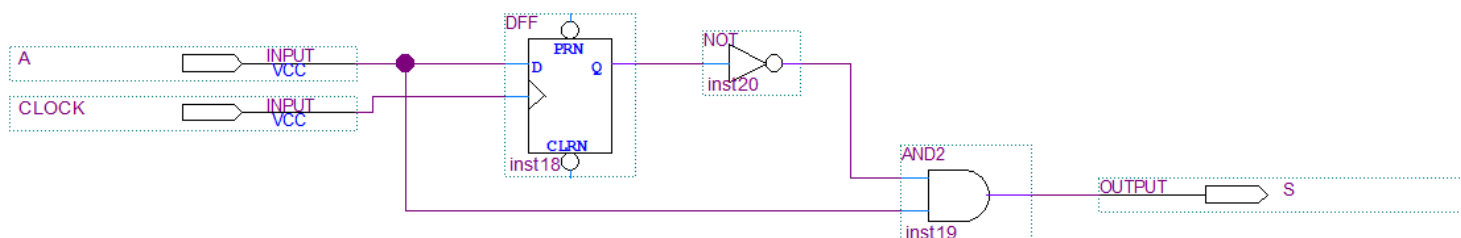
Luego, conectamos todos los EQ[i] (de los 12 bits) a una AND de 12 entradas.

Esto da como resultado:

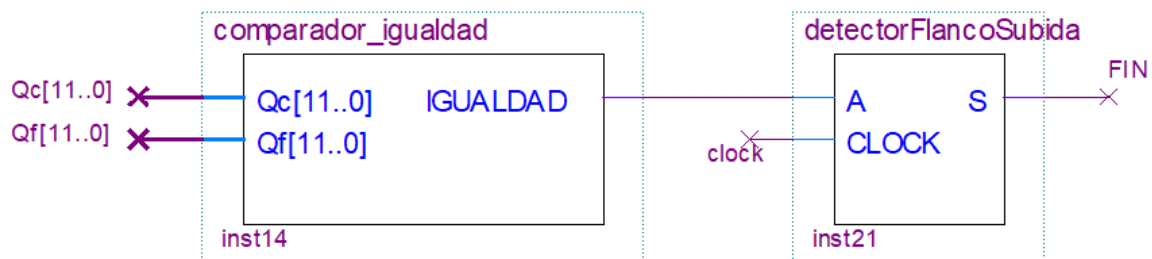
$$\text{FIN} = \text{EQ}[11] \text{ AND } \text{EQ}[10] \text{ AND } \dots \text{ AND } \text{EQ}[0]$$

Si todos los bits son iguales, entonces FIN = 1. Esta señal debe activarse por un ciclo de reloj y luego desactivarse.

Para eso usamos un detector de flancos:



Finalmente quedaría:



De esta manera si IGUALDAD pasa de 0 a 1, o sea, en el flanco ascendente, la señal FIN se activa solo por un ciclo de reloj aunque IGUALDAD se mantenga en 1.

Para la señal ERROR la activamos cuando hay una condición inválida en el circuito, es decir:

Si UP_DOWN = 0 (cuenta hacia arriba)

Debe cumplirse -> VALOR_INICIAL < VALOR_FINAL

Porque no se puede contar hacia arriba si el valor final es menor que el inicial.

Si UP_DOWN = 1 (cuenta hacia abajo)

Debe cumplirse -> INICIAL > FINAL

Porque no se puede contar hacia abajo si el final es mayor que el inicial.

Entonces debemos implementar:

- Comparador de magnitud de valores de 12 bits que evalúe $A > B$ y $A < B$.
- Según el valor de /UP/DOWN debemos elegir una de las comparaciones

-> Si /UP/DOWN = 0 (UP): activar ERROR si VALOR_INICIAL > VALOR_FINAL.

-> Si /UP/DOWN = 1 (DOWN): activar ERROR si VALOR_INICIAL < VALOR_FINAL.

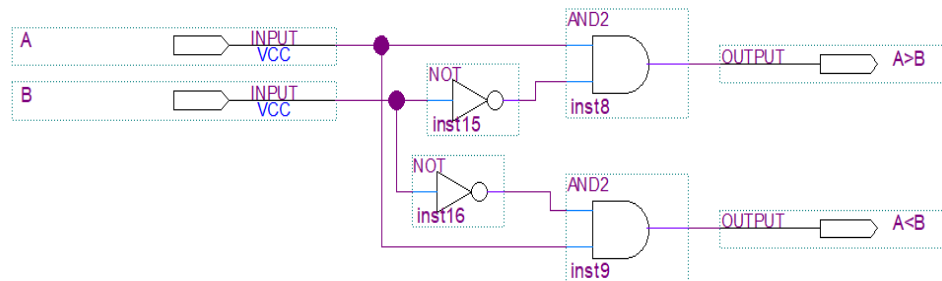
Para el comparador de 12 bits decidimos hacerlo haciendo uso de 3 comparadores de 4 bits.

Comparador de 1 bit:

A	B	A>B	A<B
0	0	0	0
0	1	0	1
1	0	1	0
1	1	0	0

$$A > B = A \cdot \overline{B}$$

$$A < B = \overline{A} \cdot B$$



En un comparador de 4 bits

el procedimiento es el siguiente:

-A>B

Recorremos todos los bits de A y B desde el mas significativo hasta que A sea 1 y B sea 0.

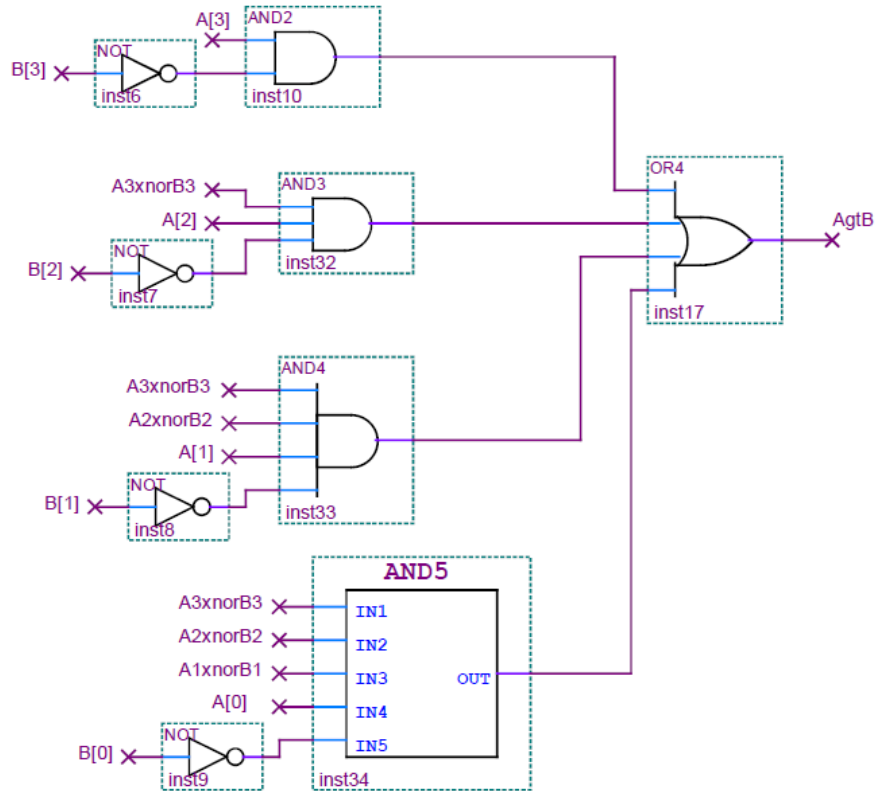
- A<B

Recorremos todos los bits de A y B desde el mas significativo hasta que A sea 0 y B sea 1.

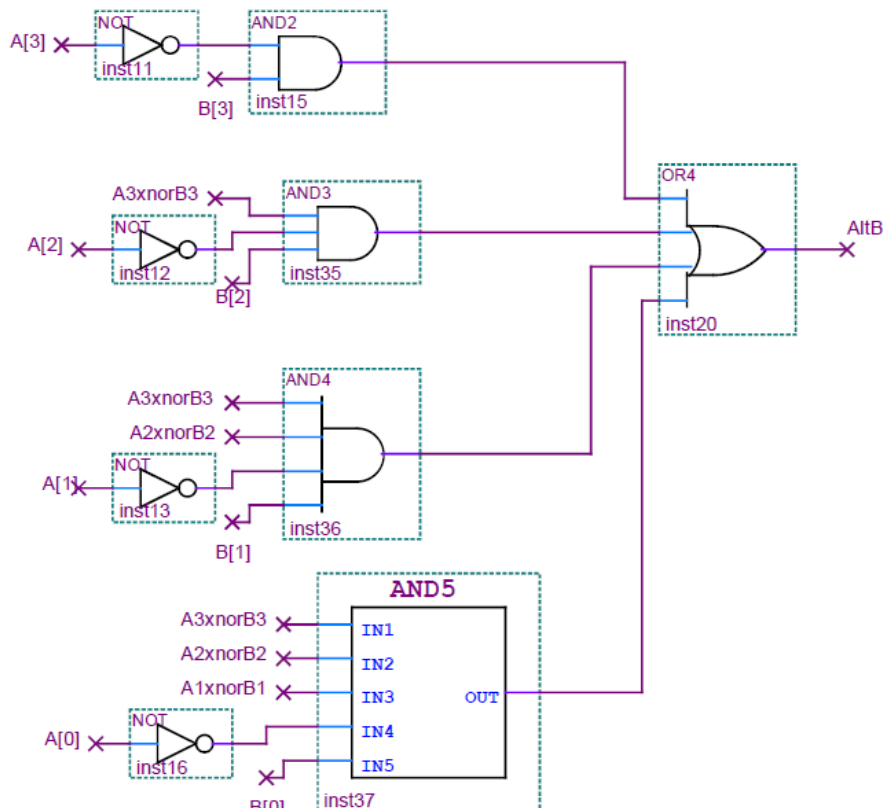
Si no ocurre ninguna de las dos los números son iguales.

Sabiendo que se compara desde el más significativo al menos significativo, las expresiones lógicas del comparador son las siguientes:

$$A > B = A_3 \cdot B_3 + (A_3 \text{ XNOR } B_3) \cdot A_2 \cdot B_2 + (A_3 \text{ XNOR } B_3) \cdot (A_2 \text{ XNOR } B_2) \cdot A_1 \cdot B_1 + (A_3 \text{ XNOR } B_3) \cdot (A_2 \text{ XNOR } B_2) \cdot (A_1 \text{ XNOR } B_1) \cdot A_0 \cdot B_0$$



$$A < B = \neg A_3 \cdot B_3 + (A_3 \text{ XNOR } B_3) \cdot \neg A_2 \cdot B_2 + (A_3 \text{ XNOR } B_3) \cdot (A_2 \text{ XNOR } B_2) \cdot \neg A_1 \cdot B_1 + (A_3 \text{ XNOR } B_3) \cdot (A_2 \text{ XNOR } B_2) \cdot (A_1 \text{ XNOR } B_1) \cdot \neg A_0 \cdot B_0$$



Teniendo los comparadores de 4 bits, los usamos para extender su funcionamiento a 12 bits.

Dividimos los números A[11...0] y B[11..0] en 3 grupos de 4 bits, del más significativo al menos:

$A = [A_{11} A_{10} A_9 A_8] \leftarrow A_H \text{ (MSB)}$
 $[A_7 A_6 A_5 A_4] \leftarrow A_M$
 $[A_3 A_2 A_1 A_0] \leftarrow A_L \text{ (LSB)}$

$B = [B_{11} B_{10} B_9 B_8] \leftarrow B_H \text{ (MSB)}$
 $[B_7 B_6 B_5 B_4] \leftarrow B_M$
 $[B_3 B_2 B_1 B_0] \leftarrow B_L$

Vamos a hacer 3 comparadores de 4 bits:

- Comparador alto (H) compara A_H con B_H
- Comparador medio (M) compara A_M con B_M
- Comparador bajo (L) compara A_L con B_L

Cada comparador de 4 bits genera tres salidas:

GT $\rightarrow A > B$

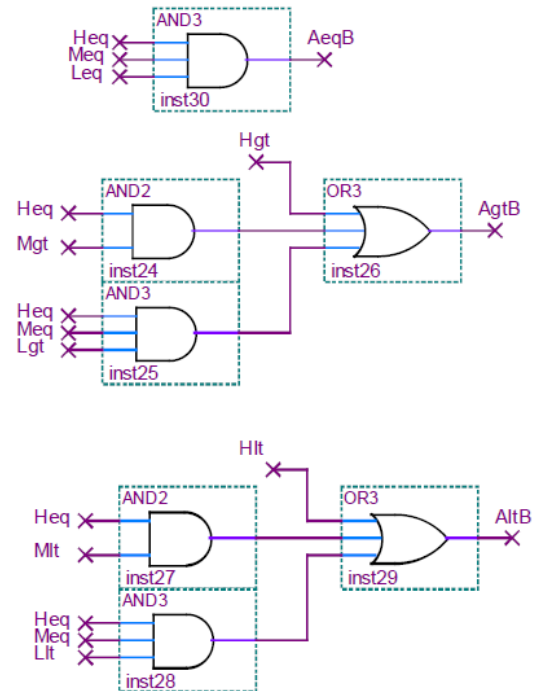
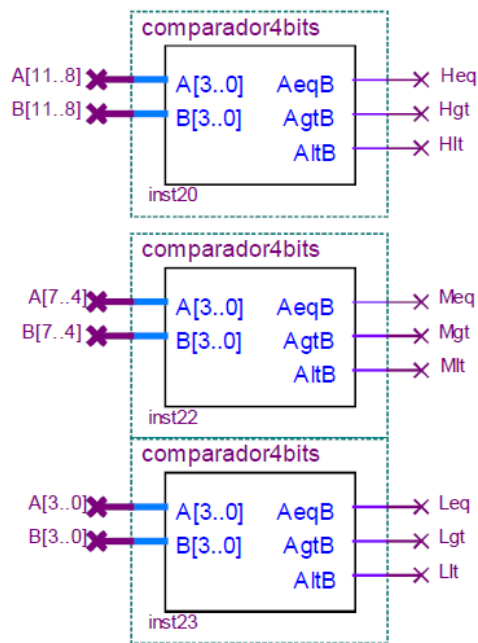
EQ $\rightarrow A = B$

LT $\rightarrow A < B$

Como en un comparador encadenado, usamos esta lógica jerárquica:

$A > B = (H_GT)$
 $\text{OR } (H_EQ \text{ AND } M_GT)$
 $\text{OR } (H_EQ \text{ AND } M_EQ \text{ AND } L_GT)$

$A < B = (H_LT)$
 $\text{OR } (H_EQ \text{ AND } M_LT)$
 $\text{OR } (H_EQ \text{ AND } M_EQ \text{ AND } L_LT)$



MÁQUINA DE ESTADOS

Para el correcto funcionamiento del contador necesitaremos una máquina de estados para controlar el flujo del circuito. Usaremos los siguientes estados:

Estado	Código	Descripción
INICIO	000	Estado inicial. Se espera la carga de valores
CARGA	001	Carga de los valores inicial y final (LOAD_I y LOAD_f)
ESPERA	010	Espera START=1
CONTEO	011	Contador activo
FIN	100	Se llegó al valor final. Espera valor de CONTINUAR

Las transiciones entre los estados dependen de señales externas a la maquina, se proponen las siguientes transiciones:

- INICIO → CARGA
 - Se pasa automáticamente al encender.
- CARGA
 - Si LOAD_I = 1 y LOAD_F = 1 → pasa a ESPERA.
 - Si alguno no se carga → permanece en CARGA.
- ESPERA

- Si **START = 1** y **b ≠ 000** → pasa a **CONTEO**.
- Si **b = 000** → permanece en **ESPERA**.
- **CONTEO**
 - Si **valor actual ≠ valor final** → permanece en **CONTEO**.
 - Si **llega al valor final** → pasa a **FIN**.
- **FIN**
 - Si **CONTINUAR = 1** → vuelve a **CARGA**.
 - Si **CONTINUAR = 0** → permanece en **FIN** (pausa).

Salidas de control:

Señal	Activa en	Función
CARGA_I	CARGA	Carga DATA en registro inicial si LOAD_I=1
CARGA_F	CARGA	Carga DATA en registro final si LOAD_F=1
HABILITAR_CNT	CONTEO	Habilita el contador para que sume o reste
FIN_OUT	FIN	Genera pulso de 1 ciclo cuando llega al valor final
SEL_DATA	CONTEO,ESPERA,FIN	Muestra el valor del contador en DATA si LOAD_I y LOAD_F estan inactivos
REINICIAR	FIN ^ CONTINUAR=1	Reinicia el ciclo

Tabla de transición de estados:

$Q_2 \ Q_1 \ Q_0$	LOAD_I	LOAD_F	START	CONTINUAR	IGUAL	B_NULO	$Q_2^+ \ Q_1^+ \ Q_0^+$	Descripción
000	X	X	X	X	X	X	001	Pasa automática mente
001	0	X	X	X	X	X	001	Espera carga de valor inicial
001	1	0	X	X	X	X	001	Espera carga de valor final
001	1	1	X	X	X	X	010	Cargas realizadas
010	X	X	0	X	X	X	010	Espera START
010	X	X	X	X	X	1	010	b=000 no cuenta
010	X	X	1	X	X	0	011	Comienza el conteo
011	X	X	X	X	0	0	011	No llegó al final, sigue contando
011	X	X	X	X	1	X	100	Terminó el conteo
100	X	X	X	0	X	X	100	Permanece en FIN(pausa)
100	X	X	X	1	X	X	001	Reinicia desde la carga de valores

Observando la tabla de estados obtenemos las siguientes expresiones lógicas:

$$Q_2^+ = Q_1 * Q_0 * IGUAL + Q_2 * \sim CONTINUAR$$

$$Q_1^+ = Q_0 * LOAD_I * LOAD_F + Q_1 * \sim START + Q_1 * START * \sim B_NULO + Q_1 * Q_0 * \sim IGUAL * \sim B_NULO$$

$$Q_0^+ = \sim Q_2 * \sim Q_1 * \sim Q_0 + Q_0 * \sim LOAD_I + Q_0 * LOAD_I * LOAD_F + Q_1 * START + Q_1 * Q_0 + Q_2 * CONTINUAR$$

Tabla de salidas por estado:

$Q_2 \ Q_1 \ Q_0$	CARGA_I	CARGA_F	HABILITAR_CNT	FIN_OUT	SEL_DATA
000	0	0	0	0	0
001	LOAD_I	LOAD_F	0	0	0
010	0	0	0	0	1
011	0	0	1	0	1
100	0	0	0	1	1

- CARGA_I y CARGA_F dependen de los botones LOAD_I y LOAD_F.
- SEL_DATA permite que el valor del contador se vea en el bus DATA cuando no se está cargando un valor.

Observando la tabla de salidas obtenemos las siguientes expresiones lógicas:

- $CARGA_I = \sim Q_2 * \sim Q_1 * Q_0 * LOAD_I$
- $CARGA_F = \sim Q_2 * \sim Q_1 * Q_0 * LOAD_F$
- $HABILITAR_CNT = \sim Q_2 * Q_1 * Q_0$
- $FIN_OUT = Q_2 * \sim Q_1 * \sim Q_0$
- $SEL_DATA = \sim Q_2 \ Q_1 \ \sim Q_0 + \sim Q_2 \ Q_1 \ Q_0 + Q_2 \ \sim Q_1 \ \sim Q_0 = \sim Q_2 \ Q_1 + Q_2 \ \sim Q_1 \ \sim Q_0$